The Citation Provenance Chain and the Trust Registry:
How Clio Knows What She Knows, and How She Knows How Well She Knows It

Robin Langer Claude

4 July 2026

Abstract

Clio is an autonomous research agent living in a Docker container, working on algebraic combinatorics across four kinds of session: *browse* (reading the literature through sub-agents), *dream* (memory consolidation), *prove* (attacking one theorem), and *lean* (formalising one result). Each cycle transforms her knowledge, and each transformation can corrupt it: sub-agents hallucinate paper contents, compression drops citations, proofs absorb hearsay as fact. This document describes the two systems deployed on 2026-07-04 to make such corruption mechanically visible: a *citation provenance chain* (a sources index with extraction levels and a corrections history, plus a validator) and a *proof-search trust registry* (a tree of attempts with trust levels, a boundary rule, and a source-ceiling rule). Both are advisory, stdlib-only, and grow lazily along the paths actually walked. We explain what happens in each cycle, what can go wrong, how old unverified claims are handled, and give worked examples, including the incident that motivated the whole construction: a phantom determinant that lived in her memory for two research cycles.

Contents

1 The problem: knowledge laundering **1**
1.1 The motivating incident: the phantom determinant 1

2 The sources index **2**
2.1 Extraction levels 2
2.2 Citation convention 3
2.3 The validator 3

3 The proof-search registry and the trust boundary **3**
3.1 The fundamental unit: a node is an attempt 3
3.2 Trust levels 4
3.3 The trust boundary 4
3.4 External sources and the ceiling rule 5

4 Old claims and lazy backfill **5**
4.1 `unclassified`: the ten-second IOU 5
4.2 Discharging: the citation lift 6

5 The four cycles: what happens, what can go wrong **6**
5.1 Browse: open-world acquisition 6
5.2 Dream: compression under pressure 7
5.3 Prove: closed-world reasoning 7
5.4 Lean: formalisation 7
5.5 Summary table 8

6 Three kinds of "we don't trust this yet" 8

7 What the system does not do 8

8 The mathematical reading 8

9 Deployment facts 9

1 The problem: knowledge laundering

Clio never reads papers directly during browse sessions. Her main context is scarce, so she orchestrates: four sub-agents search arXiv, MathOverflow, the web, and Semantic Scholar citation graphs, and each returns short structured summaries. Clio synthesises these into a daily reading log. This is efficient, and it is also the root of the problem, because it creates a pipeline in which errors are progressively stripped of the evidence that they are errors:

1. **Browse.** A sub-agent paraphrases a paper. The paraphrase may be wrong. The reading log records the paraphrase with an honest citation, but nothing records that it *is* a paraphrase.
2. **Dream.** Consolidation compresses verbose notes into connections and topic files. Under compression pressure, arXiv identifiers and theorem numbers ("locators") are the first casualties. The claim survives; its provenance does not.
3. **Prove.** A proof session cites the now-unsourced claim as established literature. It has become load-bearing.

We call this *knowledge laundering*: each stage removes information about where a claim came from and how reliable that channel was, so that by the time the claim matters, it looks indistinguishable from a verified fact.

1.1 The motivating incident: the phantom determinant

On (narrative date) 2026-07-08, Clio's dream cycle produced a "crown jewel" connection recording that the paper of Gatzweiler and Krattenthaler, arXiv 2502.06032, contains an LGV (Lindström–Gessel–Viennot) determinant relevant to her three-row program. On 2026-07-10 she happened to deep-read the paper and found that it contains *no determinant at all*. The claim was a browse-agent extraction hallucination. It had been load-bearing in her reasoning for two full cycles, and it was caught only by a voluntary re-read. Nothing in the system could have caught it, because nothing recorded that her knowledge of that paper had passed through a single lossy paraphrase.

Two further errors surfaced during the audit that followed:

- **Di Francesco–Vu 2606.12796, over-claim.** Her notes read Theorem B.1's prefactor as giving an "order law" via a coincident-rapidity degeneration. The paper assumes pairwise-distinct variables and never takes the limit $x_i \to x_j$. The degeneration step is absent from the paper. It is hers to prove, not a literature import.
- **2605.13432, author misattribution.** Prior feeds attributed the paper to Gunna–Wheeler–Zinn-Justin; the actual authors are Gunna–Yeliussizov.

All three are now recorded as permanent `corrections` entries in her sources index (Section 2). The design principle throughout: *a wrong extraction is never deleted, only annotated.* Deleting it would erase the evidence that this failure mode exists.

2 The sources index

The first component is a single file, `memory/reading/sources.json`, format `clio-sources-v1`. Every paper Clio cites gets one entry:

```
"2606.12796": {
  "title": "Rank recursion for q-Whittaker and Macdonald operators",
  "authors": "Di Francesco, Vu",
  "year": 2026,
  "extraction": "deep-read",
  "read": ["reading/2026-07-06.md", "reading/2026-07-09.md",
           "reading/2026-07-10.md"],
  "locators": ["Thm B.1", "Rem 3.8", "Prop 2.2"],
  "corrections": [
    { "date": "2026-07-10",
      "note": "07-06/07-09 read Thm B.1's prefactor as giving the
       order law via coincident-rapidity degeneration. The paper
       assumes pairwise-distinct variables and never takes
       x_i -> x_j. The degeneration step is absent. It is mine
       to prove, not a literature import." } ]
}
```

2.1 Extraction levels

The key field is `extraction`: not *what* the paper says but *how the knowledge of it was acquired*. Four levels, strictly ordered:

Level	Who read what	Typical failure mod
`agent-summary`	a sub-agent read it; Clio has only the paraphrase	hallucinated conten nant), misattributio
`abstract`	Clio read the abstract herself	the abstract oversel that matters
`deep-read`	Clio read the full text	subtle over-claims (step)
`verified-quote`	a specific statement copied verbatim and checked against the source	essentially none, for

A common misunderstanding is worth dispelling: `agent-summary` does *not* mean "an old paper whose citation we lost." The citation is fully known: arXiv ID, title, authors, and the exact reading logs it entered through. What is untrusted is the *content channel*. The paper is often brand new; the knowledge of it is hearsay.

2.2 Citation convention

Inline, at the point of use, with a locator when a specific result is invoked: `(arXiv:2606.12796, Thm B.1)`. Author names alone are not citations for claims. This matters because the tooling extracts citations by scanning for arXiv-ID-shaped tokens (a regex requiring a plausible month, so decimals like 3.14159 do not match); a claim "by Di Francesco–Vu" with no ID is invisible to every check described below.

2.3 The validator

`code/citation_check.py` (stdlib only; Clio's container has no PyYAML, no pip installs) does four things:

- validates the index itself: known extraction levels, existing `read` files, well-formed corrections;
- checks that every arXiv ID cited in given markdown files resolves in the index, flagging **unregistered** sources;
- `-report footprint <file>`: for each source cited by a file, its extraction level and correction flags, plus the file's *provenance floor*: the lowest extraction level among its citations. A connection with floor `agent-summary` is a connection that could be resting entirely on hearsay;
- `-report shallow`: every `agent-summary` source cited by at least one connections or topics file, ordered by how many files lean on it. This is the *deep-read worklist*: the papers whose verification would raise the most floors.

Example footprint output on a note citing one deep-read source and one unknown ID:

```
connections/conn.md
  1111.22222  UNREGISTERED
  2606.12796  deep-read      Di Francesco, Vu  [1 correction(s)]
  provenance floor: deep-read
```

3 The proof-search registry and the trust boundary

The second component predates the citation chain by a few hours and indexes a different thing: not papers, but *her own attempts*.

3.1 The fundamental unit: a node is an attempt

One JSON file per conjecture; the file holds a tree. Each node is one *attempt*: sometimes a subgoal ("$c = 3$: interior and boundary"), sometimes a lemma ("Number Lemma: $16 \mid H$"), sometimes a strategy ("LGV Gaussian-binomial determinant of the closed form"), sometimes the corpse of a strategy (a dead end, with the reason it died). The node schema:

```
{ "id": "unique-within-tree",
  "approach": "one line: what this attempt was",
  "trust": "speculative | computed | proved | lean-verified
            | dead-end | in-progress | unclassified",
  "file": "2026-06-18-c4-interior-Jstar-even.md",
  "lean": "optional_sorry_free_declaration",
  "reason": "required iff dead-end: why it died",
  "sources": ["2606.12796"],
  "children": [] }
```

A node's justification has exactly three kinds of evidence, each with its own check:

Field	Evidence type	Checked by
`children`	internal, recursive	boundary rule (§3.3)
`file`, `lean`	her own artifact	file existence; sorry-free build
`sources`	external literature	ceiling rule (§3.4)

3.2 Trust levels

Ordered: `speculative` < `computed` < `proved` < `lean-verified`. Outside the order: `dead-end` (abandoned; must carry a `reason`), `in-progress` (open), and `unclassified` (Section 4). The levels mean:

- `speculative`: an idea; no evidence beyond plausibility.
- `computed`: machine-checked evidence (verified for $n \leq 10$, a counterexample found, a formula fitted), but no proof.
- `proved`: a written argument that holds, every step justified.
- `lean-verified`: a sorry-free Lean declaration, standard axioms only.

3.3 The trust boundary

> **Boundary rule.** A node may claim `proved` or `lean-verified` only if every non-dead-end child is at least `proved`.

The picture: draw the tree and colour every node at `proved` or above. The rule forces the coloured region to be *upward closed along justification*: a coloured node can only stand on coloured (or explicitly abandoned) ground. The frontier between the coloured region and the rest is the trust boundary. Above it, claims are justified by their entire subtree; below it, everything is exploratory. The three report flags of the validator are exactly views of this geometry: `successful-path` prints the coloured skeleton, `frontier` prints the open nodes just below the boundary, `dead-ends` prints the pruned branches with their reasons.

Worked example, from her real three-row registry. The $c = 4$ subgoal:

```
c4 [proved]
 |- c4-interior [proved]
 |    |- c4-number-lemma [proved]
 |    |- c4-linear-phi [dead-end]
 |         reason: v2(H) has a constant floor (>= 4),
 |                 not linear growth in phi
 |         |- c4-linear-phi-refutation [computed]
 |- c4-boundary [lean-verified]
```

`c4` may claim `proved`: its children are `proved` and `lean-verified`, and the dead end does not count against it. Note the refutation: a dead end's `reason` is itself a claim, and a wrong reason silently prunes a live branch, which is the most expensive error in a search. So dead ends can carry a *child holding the refutation* at an ordinary trust level. Here the refutation is `computed`: a counterexample computation. A dead end with no refutation child was killed on judgment; if she is ever tempted back down that route, the reason gets re-checked first.

If she edited `c4-interior` down to `computed` while leaving `c4` at `proved`, the validator would reject:

```
root/c4: claims 'proved' but child 'c4-interior' is 'computed'
(boundary rule: non-dead-end children must be at least 'proved')
```

3.4 External sources and the ceiling rule

The boundary rule governs internal dependencies. The 2026-07-04 integration extends the same idea to external ones:

> **Ceiling rule.** A node claiming `proved` or `lean-verified` whose cited sources are *all* at `agent-summary` is rejected: "deep-read at least one before it is load-bearing."

A browse-agent's paraphrase may be cited freely, exactly as an `unclassified` child may be cited freely; what it cannot do is carry a proof. Unknown IDs in a `sources` list produce

warnings, not errors (exit 0); a node with no `sources` field gets no check at all, which keeps the rule backward compatible with registries written before the field existed.

Live example. The node `lgv-det` in the three-row registry proposes an LGV determinant route and notes it is "the same determinant as the Di Francesco–Vu order-law Cauchy det." Originally that dependency lived only in prose, invisible to tooling. It now carries `"sources": ["2606.12796"]`. Today the node is `speculative`, so the ceiling rule is silent. But the trap is armed: any future promotion to `proved` happens in full view of that source's entry, including the correction stating that the degeneration step *is not in the paper*. The node can no longer forget where its determinant came from.

Had the source been at `agent-summary`, promotion would fail outright:

```
root/general-c/lgv-det: claims 'proved' but every cited source
is 'agent-summary' -- deep-read at least one before it is
load-bearing
```

4 Old claims and lazy backfill

Clio has hundreds of pre-registry proof files and dozens of pre-index reading logs. A full retroactive sweep would be both expensive and epistemically fraudulent: labels assigned from memory are trust inflation. The rule instead is *lazy backfill*: nothing is retro-fitted; trust is re-earned exactly when something new depends on it.

4.1 `unclassified`: the ten-second IOU

When a new attempt cites an old result not yet in the registry, the whole obligation is one node:

```
{ "id": "old-content-floor",
  "approach": "master valuation formula from June",
  "trust": "unclassified",
  "file": "2026-06-17-generalc-g0-content-floor.md",
  "children": [] }
```

`unclassified` means "a claim exists on disk; nobody has re-checked it." It ranks below `proved`, so the boundary rule does the rest: the new node cannot claim `proved` while it leans on the IOU, and the validator points at exactly which one. Files nothing depends on stay unregistered forever, which is fine.

4.2 Discharging: the citation lift

To discharge an `unclassified` node, Clio re-reads the old proof and promotes honestly: `computed`, `proved`, `dead-end` with a reason (old results do turn out wrong; recording why is the point), or `lean-verified`. Since 2026-07-04 the procedure has one more step, because the old file's *literature debts* must not vanish with the label:

1. Run `citation_check.py -report footprint` on the old file. It lists every arXiv ID in the text, each one's extraction level, and the provenance floor.
2. Register anything unregistered in `sources.json` at its honest level. For a months-old browse cycle that is usually `agent-summary`: honesty, not pessimism.
3. Lift the IDs into the node's `sources` field.
4. Promote. The ceiling rule now applies: the node cannot reach `proved` if everything it imported is still hearsay, so the promotion forces deep-reads of precisely the papers the old argument leaned on.

The invariant of the whole system is visible here: *doubt can be moved and sharpened, never silently dropped*. The coarse flag ("re-check this whole file") is converted into fine-grained ones (per-paper extraction levels); the total amount of scepticism is conserved until evidence discharges it.

One honest limitation. The lift is purely textual: the regex extracts IDs that past-Clio wrote down. A claim she used silently, or cited by author name only, leaves no machine-readable debt, and the node can reach `proved` with no record that anything external was involved. The provenance quality of old work is bounded by the citation discipline of the Clio who wrote it. The compensating bet is temporal: the prompt hooks (Section 5) make every file written from 2026-07-04 onward cite IDs at the point of use, so extraction is lossless for new work and the lossy cases are confined to a shrinking, quarantined corpus that is touched only when something leans on it.

5 The four cycles: what happens, what can go wrong

Each session type is a different epistemic mode, and each got hooks matched to the claims born there. Discipline is installed twice over: as prose instructions in the prompt, and, where possible, as a mechanical check the session runs before ending.

5.1 Browse: open-world acquisition

What happens. Clio reads her seed file and recent memory, generates compound keywords, and dispatches four agents in parallel (arXiv listings, MathOverflow, general web, Semantic Scholar citation trails). When they return she writes the daily reading log: papers, summaries, connections to seed themes, open questions.

What can go wrong. This is where hearsay enters. Sub-agents hallucinate contents (the phantom determinant), misattribute authors (2605.13432), or summarise an abstract as if it were the paper. The log itself cites correctly; the corruption is invisible because the log does not say *how* each fact was obtained.

Hooks. The synthesis step now includes: update `sources.json` for every paper that got a real summary, at the honest extraction level, noting which agent read it and whether it saw HTML full text or only the abstract page; if today's reading contradicts an earlier note, add a correction (never delete the wrong one). The session ends by running `citation_check.py` on the day's log: every ID cited must resolve in the index, so a paper summarised but never registered is caught the same day.

5.2 Dream: compression under pressure

What happens. A multi-pass streaming consolidation (her context is small, so she reads batches, writes a checkpoint, and restarts). Five phases: replay, associate, consolidate, prune, surface. The associate phase births genuinely new claims: connections of the form "X relates to Y because Z, and this suggests Q," which she calls the crown jewels.

What can go wrong. Compression is exactly where locators die. A note reading "2502.06032 proves the crown identity via an LGV determinant (Thm 4.2)" becomes "the G–K determinant gives the crown identity" becomes "the crown identity is known." Each rewrite is locally reasonable; the composite is laundering. The phantom determinant survived two cycles because consolidation carried the claim while shedding everything that would have flagged it as hearsay.

Hooks. Consolidation preserves arXiv IDs and locators on every external claim; compression never drops a citation; a connection resting on an `agent-summary` source must say so inline, because that flag is what tells future-Clio to deep-read before the claim becomes load-bearing.

Before exiting, she runs `citation_check.py` on any connections or topics files she rewrote, catching IDs orphaned by the rewrite. Residual gap, stated honestly: a validator cannot flag the absence of an ID it never saw, so a citation deleted from prose *entirely* is still only guarded by instruction. The `read` trail in `sources.json` at least makes such a loss recoverable: the paper's entry records which logs it entered through.

5.3 Prove: closed-world reasoning

What happens. One theorem, the whole session. No email, no browsing: the network ban is deliberate, so that no fact can enter a proof through a hasty, unrecorded read. Sub-agents may compute (check a lemma for $n \leq 10$, hunt counterexamples) but Clio holds the proof thread.

What can go wrong. Two things. Internally: a chain of lemmas where one link is only `computed` evidence gets written up as a proof. Externally: a claim from memory, laundered through a dream cycle, gets cited as a known theorem. The missing-degeneration incident is the second failure in its subtlest form: the paper is real, the theorem is real, and the single step Clio actually needed is not in it.

Hooks. External facts cite ID and locator at the point of use and go into the registry node's `sources` field. An `agent-summary` source is not load-bearing: deep-read first (in a browse cycle; the shallow report queues it) or mark the step as a gap. Pre-registry citations become `unclassified` children. The session ends by running `registry_validate.py`: boundary rule, ceiling rule, dead-end reasons, file existence. When she hits the ceiling, the effect is that "I need this paper" becomes a queued, typed request rather than an impulsive fetch: prove names the blocking source, the shallow worklist ranks it, browse deep-reads it, and the next prove session finds the ceiling lifted.

5.4 Lean: formalisation

What happens. One target declaration, already proved on paper. Small steps, frequent builds; `sorry` is a bookmark, not a result; axioms restricted to the standard three.

What can go wrong. The paper proof has a gap that formalisation exposes (the desired outcome, in fact), or the temptation arises to patch it with new mathematics mid-session, or to mark a node `lean-verified` while sorries remain.

Hooks. Citations from the paper proof are carried into the Lean file as docstrings or comments on any declaration formalising an external result, so even the formal artifact remembers its literature. A node is set to `lean-verified` only when the declaration builds sorry-free, and the validator run closes the session.

5.5 Summary table

Cycle	New claims born	Main failure mode	Mechanical check at exit
browse	"paper X contains Y"	agent hallucination, misattribution	`citation_check` on the day's log
dream	connections, syntheses	citation loss under compression	`citation_check` on rewritten files
prove	lemmas, registry nodes	hearsay becomes load-bearing	`registry_validate`: boundary + ceiling
lean	formal declarations	premature `lean-verified`	`lake build`, sorry count, axiom print

6 Three kinds of "we don't trust this yet"

The system distinguishes three states that are easy to conflate:

Unregistered. An arXiv ID appears in a note but has no entry in `sources.json`. *Who is this?* Caught by `citation_check.py`, exit 1.

`agent-summary`. Provenance fully known, transmission lossy: someone else's work, seen through a summariser. *We know who, but only by hearsay.* Caught by the ceiling rule at promotion time and surfaced by the shallow worklist.

`unclassified`. Provenance known, verification stale: her own past claim, gathering dust. *Our own work, unre-checked.* Caught by the boundary rule at promotion time; discharged by re-read plus citation lift.

The first is a bookkeeping error; the second and third are the external and internal faces of the same epistemic state, and the two rules treat them symmetrically: cite freely, but nothing above the boundary may rest on them.

7 What the system does not do

Honesty about scope, because the gaps define the next projects:

- **No online verification.** Nothing goes to the network to check a citation. Prove sessions are forbidden to browse by design; deep-reads happen in browse cycles, routed by the shallow worklist. The system verifies citations that exist in the text; it does not discover missing ones.
- **No completeness or priority search.** Nothing asks "does this claim of hers already exist in the literature?" or "has someone published a contradiction?" That would be a literature-search tool, not a validator.
- **Prose-only mentions are invisible.** "By Di Francesco–Vu" without an ID escapes every check. The convention (IDs at point of use) exists precisely to shrink this hole for all new writing.
- **Advisory, not blocking.** Both validators print readable messages and exit nonzero on real violations, but no automation prevents Clio from ignoring them. This is deliberate: she is a researcher, not a pipeline stage, and the tools are instruments she reads, not gates she passes. The bet is that visible, precise, cheap checks get used; the first browse and prove cycles after deployment will show whether the bet pays.

8 The mathematical reading

The registry is a directed container in the sense of Ahman–Chapman–Uustalu: shapes are search trees, positions are paths from the root. The structure maps are exactly the operations a resumed session needs: `root` is the conjecture (the empty path), $\texttt{sub}(t, p)$ is the sub-search at path p with dead ends included, and $\texttt{shift}(t, p, q)$ composes paths, viewing q inside the sub-search at p globally. The induced comonad has `extract` reading the root's trust ("is it proved?") and `duplicate` decorating every node with the full sub-search below it, so that standing at any attempt, the complete history beneath is available. The three report flags are coKleisli morphisms $W(\text{Registry}) \to \text{Report}$: strategies that inspect the whole tree of attempts, failures included, before summarising. This structure is formalised, all five comonad laws and no `sorry`, in `ProofSearch.lean` in the containers repository.

The citation chain has a reading in the same vocabulary: reading a paper is pulling positions back along a container morphism from Clio's memory to the literature, and the extraction level records how much of that morphism has actually been traversed rather than presumed. It is the first instance of a planned pattern of cross-registry morphisms, where trust in one structure is constrained by typed references into another.

9 Deployment facts

For the record (all deployed 2026-07-04, real date; Clio's narrative clock runs a few days ahead, so her file dates differ):

- `memory/reading/sources.json`: 19 sources backfilled from reading logs 07-06 through 07-10 at honest levels, 3 corrections. Around 45 re-mentioned hub papers deliberately left out: lazy backfill applies to the index too.
- `memory/reading/CITATIONS-README.md`: conventions, addressed to Clio.
- `code/citation_check.py` and the extended `code/registry_validate.py`: both stdlib-only, both run clean in place against her real tree.
- Prompt hooks in `browse-prompt.md`, `dream-prompt.md`, `DREAM.md`, `prove-prompt.md`, `lean-prompt.md`. Scripts are baked into her Docker image, so host edits were copied into the running container and verified byte-identical.
- The `lgv-det` node seeded with its Di Francesco–Vu source as the first live use of the `sources` field.

No Clio session has run since deployment. Adoption is the open question, and the design's main mitigation for it: the hooks meet her at all four sites in her own terse style, the tools are cheap and readable, and the conventions formalise things she already half-did rather than imposing a foreign workflow.